

P2434R2: Nondeterministic pointer provenance

Audience: EWG, CWG, SG22

S. Davis Herring <herring@lanl.gov>

Los Alamos National Laboratory

October 14, 2024

History

Since [r1](#):

- Rebased onto N4988
- Explained interaction with P1494/WG14:N3128
- Gave [basic.types.trivial] a title
- Removed term “bit value”, relying more on “value representation”
- Clarified unspecified pointer comparisons
- Extended implementation effects discussion

Since [r0](#):

- Rebased onto N4981
- Clarified discussion of P2318R1’s analysis and proposal
- Discussed implications for pointer zap

Introduction

[P2318R1](#) describes a variety of plausible models of pointer provenance that differ principally in how they handle conversions between pointers and integers (including the integer values of the storage bytes for a pointer). (See its §A.4 for discussion of the variants in terms of examples.) It proposes the variant called PNVI-ae-udi, which does seem to provide a good combination of optimization possibilities and support for existing code. However, in several cases it is overly charitable to the programmer: for example, using an explicit copy loop rather than calling `std::memcpy` “exposes” storage, which can interfere with optimization, even if the byte values obviously do not escape. (The paper proposes to eventually add annotations to avoid such unwanted side effects.) It also sometimes depends in an apparently arbitrary fashion on the order of operations with no data dependencies (as in the `pointer_from_integer_1ig.c` example with an exposure of `j` added).

The main alternative that was considered and rejected is the PVI model, which avoids the notion of storage exposure but imposes further restrictions on integer conversions. These restrictions provide further opportunities for optimization but also complicate the execution model in subtle ways that make it difficult for the programmer to determine whether a manipulation preserves the validity of a pointer (yet to be reconstructed). They also interact badly with serialization of pointers where operations on the converted pointer value are entirely invisible; additional annotations might be required to support this use case.

This paper compares the existing rules for pointers to these provenance models and proposes minor changes to better align them. These changes have the additional benefit of addressing some of the concerns surrounding “pointer zap”. In particular, the use cases for the “provenance fence” operation proposed in [P2414R3](#) become implementable with only mild assumptions about implementation-defined behavior for invalid pointer values.

Analysis

Pointer values are described in abstract terms ([basic.compound]/3); while they “represent the address” of an object, that address may be given a numerical interpretation only via copying the bits into an integer or via the implementation-defined mapping to integers ([expr.reinterpret.cast]/4), and there is no specification of how any address is chosen. As such, integers obtained by memcpying or casting pointers may be taken to be completely unspecified aside from the (separate) round-trip requirements. ([basic.align]/1 talks about addresses as ordinal or cardinal numbers of bytes, but nothing else depends on that interpretation.)

This nondeterminism already implies undefined behavior in many of the circumstances that the provenance models are meant to reject. Consider the simple example

```
int main() {
    int jenny=0;
    // std::cout << std::hex << (uintptr_t)&jenny << '\n';
    *(int*)8675309=1;
    return jenny;
}
```

(Assume that the implementation does not specially define a pointer value corresponding to this particular integer.) Even if the address of jenny is the suspicious value given, this has undefined behavior under PNVI-a-udi (because the address of jenny is never exposed) and PVI (because the integer literal is not derived from the address of jenny). However, it is equally undefined in N4988 because there are possible executions of the program ([intro.abstract]/5) where the address is some other value and the cast produces an invalid pointer value. P2318R1 considers this interpretation but does not deem it conclusive, explaining that the same analysis of many tests can be obtained from address nondeterminism instead of the explicit provenance semantics but lamenting that that approach “requir[es] examination of multiple executions”.

Note that in N4988 printing the address is no help: even if the program displays “845fed”, that itself can be a manifestation of the undefined behavior. However, an interactive program that reads input after writing output (which has long been neglected by [intro.abstract]/5) constitutes a significant complication here:

```
int coin(int x,int) {return x;}
int coin() {
    int i=0;
    return coin(i,++i); // either 0 or 1 based on unspecified evaluation order
}
int main() {
    const int mystery=coin();
    std::cout << "number other than " << mystery << ": ";
    int d;
    std::cin >> d;
    return 127/(d-mystery);
}
```

It would be absurd to suggest that this program has undefined behavior if a user who was prompted not to enter 0 entered 1 just because a different execution exists where entering 1 would produce division by zero. (Consider that similar prompts might be used for indices into arrays of size learned only at runtime.) Clearly printing &jenny might influence any further input, perhaps as simple as a flag that controls using the guessed address (or an integer that switches among multiple guesses). As such, any guess that might be informed by interactive input ought to be allowed. This paper does not propose to specify appropriate semantics for interactive programs in general, but the notion of an observable checkpoint from [P1494](#) or the similar restriction of undefined behavior in C23 via [WG14:N3128](#) do something stronger by making the output already constitute a commitment by the implementation to a particular subset of potential executions. The implementation would then be obliged to either have the output of &jenny not match the guess or honor the guess; while in simple cases the output could

be adjusted, the guess would need to be honored in programs wherein the connection between the output and the guess is sufficiently complicated. The latter approach has almost exactly the same semantics as PNVI-ae-udi, except that an exposure that can be proven not to influence observable behavior can be disregarded.

Further subtle distinctions concern integer manipulation; consider

```
int main() {
    int x,y=0;
    uintptr_t p=(uintptr_t)&x,q=(uintptr_t)&y;
    p^=q;
    q^=p;
    p^=q;
    *(int*)q=*(int*)p;
    return x;
}
```

PVI disallows this manipulation, saying that the values resulting from operations on *p* and *q* do not inherit their provenances; PNVI-ae-udi allows it because it simply observes that both addresses have escaped from the abstract world of pointer values. In the latter case, the compiler would have to support “guessing” addresses even if it could prove that the value of such a pointer doesn’t actually depend on the original pointer. N4988 already handles this case more elegantly: the program has well defined behavior because for any choice of addresses for *x* and *y*, *q* (*p*) ends up being the address of *x* (*y*), so the casts back to pointers produce the swapped pointer values. This interpretation extends to arbitrary integer manipulations and I/O: the operations allowed are precisely those that *reliably* reproduce some address initially obtained, whether via arithmetic, control flow, or data dependencies. In practice it is impossible to detect every pointer value construction that fails to be reliable (that “guesses”) or fails to influence observable behavior (for interactive input), but there is no obligation to do so since the behavior is undefined in such cases.

The pointer zap rule ([basic.compound]/4) has two purposes: its footnote explains that even examining a pointer into deallocated storage might trap (in the course of loading something like a segment register), but it also serves as a sort of notional “generation counter” for reused parts of the free store. Consider

```
int main() {
    int *p=new int;
    uintptr_t i=(uintptr_t)p;
    delete p;
    p=new int(1);
    if((uintptr_t)p==i) *(int*)i=0;
    return *p;
}
```

PVI rejects this LBYL approach: *i* cannot acquire the provenance of the new *p* merely because it has been compared to another integer with that provenance. (Consider that the comparison might occur in an opaque function.) PNVI-ae-udi allows it because the address of that new object is exposed in the course of making the check; N4988 allows it simply because *i* is cast back to a pointer precisely when it has the same value as the cast of the new *p*. It is only natural that two integers with the same value should have the same behavior.

N4988 does not, however, implement the user-disambiguation (*udi*) provision: in saying that a pointer subjected to a round-trip conversion “will have its original value”, [expr.reinterpret.cast]/5 erroneously forbids real implementations that produce the same integer value for pointers to an object and one past the object that immediately precedes it in memory. Similarly, [basic.types.general]/2–3 refer to a singular value for what might be a pointer reconstructed from bytes (and /4 claims that the bits are [sufficient to determine the value](#)). [bit.cast]/2 acknowledges the possibility of more than one value with the same value representation, but leaves it unspecified which is produced.

The case of concern for lock-free algorithms is analogous to the one-past pointer situation in that multiple abstract pointer values exist whose memory representations (as must be the currency of atomic operations) are

identical. Of course, at most one such value (of a given type) is valid at any moment during execution, so there is no ambiguity as to which value the program would prefer to result from a conversion. In its place is the complication that the normal phrasings of these algorithms cannot use any such result because it might be invalidated by unsequenced deallocations. In this case a temporal version of user-disambiguation may be applied: any correct algorithm discovers that the address in question is that of some live object before it uses any associated pointer value, thereby nominating that object's identity as the value of the pointer.

Proposal

To implement *udi*, apply the same *angelic nondeterminism* by which [implicit object creation](#) selects the objects to create: if any pointer value exists that corresponds to the integer and gives the program defined behavior, one such value is the result. (As is necessary for any implementation to exist, no program can observe the acausal information about which pointer value is selected.) This change affects [basic.types.general]/2–4, [expr.reinterpret.cast]/5, and [bit.cast]/2 (which currently instead plays into the general *demonic nondeterminism* of [intro.abstract]/5). This paper does not attempt to address the situation of modifying a pointer by [storing to part of its object representation](#), but it changes the simple memcpy case to avoid eventually giving different behavior to direct and circuitous means of accomplishing a bit-wise copy.

To avoid confusing inconsistencies with comparing their integer representations (on implementations where each address has just one such), restrict [expr.eq] to provide consistent results for any pair of pointer values. This change cannot be detected during constant evaluation ([expr.const]/5.24); following P2318R1, we could consider actually comparing the addresses other than during constant evaluation (because programs cannot benefit from the inherently unreliable != result for distinct pointer values that share an address).

Consequences for pointer zap

Note that the pointer value that gives the program defined behavior might be to an object whose lifetime has not started or even whose region of storage has not yet been created. Clearly such a “prospective” pointer value cannot immediately be used: any attempt to do so necessitates that the result of the integer conversion instead be a pointer to some object in existing storage which cannot then be used after a delete and new. Of course, the reuse of the address might never take place; a correct algorithm will never use the pointer in that circumstance, so it is harmless for it to refer to the previous object. Otherwise, there is no inconsistency in allowing it to be stored until such time as its object comes into existence. (Note that references cannot be used in place of pointers here because a pointer value must be valid in the context of applying * to it ([basic.compound]/4).)

We can then implement LIFO Push with a trivial modification of the initial implementation from P2414R3:

```
template <typename Node> class LIFOList { // Node must support set_next()
    std::atomic<Node*> top_{nullptr};
public:
    void push(Node* newnode) {
        while (true) {
            Node* oldtop = reinterpret_cast<Node*>(
                reinterpret_cast<std::uintptr_t>(top_.load())); // step 1
            newnode->set_next(oldtop); // step 2
            if (top_.compare_exchange_weak(oldtop, newnode)) return; // step 3
        }
    }

    Node* pop_all() { return top_.exchange(nullptr); }
};
```

The only change here is the round-trip cast in step 1. If top_ is deallocated and replaced (at the same address) after the load(), oldtop is a pointer to the new object, so newnode contains a pointer valid when it is installed

by the compare-exchange. No “provenance fence” is then needed in `pop_all` because the pointers it reads are all valid at the time. Note that the algorithm still relies on the implementation-defined behavior of applying various operations (other than indirection or deallocation) to the pointer value `top_.load()` that might not be valid in the context of such operations ([`basic.compound`]/4), since the reallocation might not yet have happened (if it happens at all).

This paper does not propose requiring that all implementations support these operations (integer conversion, initialization/assignment, and compare-exchange) on non-valid pointer values: it is already optional for the implementation to provide `std::uintptr_t`, so it is not fundamentally more restrictive to implement LIFO Push only on implementations that support such operations. Nonetheless, EWG might choose to strengthen these requirements while considering these applications to improve their portability.

Nor does this paper propose special behavior for `std::atomic<T*>` (as has been considered as another partial solution for such algorithms): the mismatched value obtained by the compare-exchange must itself be subjected to a round-trip in order to potentially refer to a future object. It would however be reasonable to specify that such a round trip is implicit in the operation since `std::atomic` already traffics in value representations and suppresses certain kinds of undefined behavior.

Consequences for implementations

These changes are intended more to formalize than to change existing implementations, in that there does not seem to be any other consistent model that can be used in the presence of multiple pointer values with the same address. However, implementations do not always use a consistent model, as illustrated by the following program:

```
#include<stdint>
int y,x; // use x,y at -00
int main() {
    y=1;
    int *const p=&x+1,*const q=&y;
    const std::uintptr_t u=(std::uintptr_t)p,v=(std::uintptr_t)q;
    if(u==v) {
        const auto w=u+v-u/2-u/2-(u&1);
        if(w==u) { // redundant check
            *(int*)w=2;
            return y;
        }
    }
    return -1;
}
```

Here GCC and Clang each forward the store of 1 to the `return y` when optimizing, despite the fact that `w` is actually equal to `v` regardless of `u`'s value (rather than, say, equal to `u` regardless of `v`) and despite the two surrounding equality checks. Apparently the “original value” for `w` is deduced from the “redundant check” comparison, since changing that to `w==v` causes both implementations to return 2 instead. (Indeed, the generated assembly describes the store through `w` as being to `$x+4` as given and to `$y` with the comparison to `v` substituted.)

The commonly-assumed implementation-defined support for non-valid pointer values also produces what may be a surprising optimization limitation in an example from Hans Boehm:

```
void f(T* p) {
    T* t = new T();
    if (opaque_fn(t)) {
        // *t and *p can alias here
    }
}
```

It is possible that `p` is a prospective pointer value referring to `*t` that was produced by a round-trip cast (if the implementation allowed such to occur before the allocation of `t`) and that `opaque_fn` returns true only if `t` has the same address as `p` (stored elsewhere). This situation seems surprising based on local reasoning akin to that involved in [certain multi-threaded cases](#), but it is isomorphic to LIFO Push above: `p` is `oldtop` (as passed to `set_next`), `t` is `newnode`, and `opaque_fn` does the CAS. Since we want (`set_next`'s copies of) `oldtop` to be usable (without requiring something like `usable_ptr` there), we need `p` to be as usable as `t` is here.

One further point of concern raised about *udi* is that it might allow unrestricted navigation through a structure with a pointer, which (despite the existence of `offsetof`) current implementations forbid. However, such manipulations remain undefined behavior except where they do not grant any additional power: given

```
struct point {float x,y,z};
void g(float *f);
float zero(point p) {
    p.z=0;
    g(&p.y);
    return p.z;
}
```

`zero` can still perform SRA and return `0f` because `g` cannot access `p.z`: it can't obtain `&p` despite `point` being standard-layout because `y` isn't the first member, and while on practical implementations we know that `(uintptr_t)(&p.y+1)==(uintptr_t)&p.z`, `g` cannot dereference `(float*)(uintptr_t)(f+1)` without having proven that the equality holds in its specific case (since the implementation might have a complicated pointer-integer relationship that just *usually* has that property). Proving the equality in a specific case of course requires having access to `&p.z` or some quantity computed therefrom, so the usual alias analysis holds.

Wording

Relative to N4988.

#[basic.types]

#[basic.types.general]

Move paragraphs 2 and 3 to `[basic.types.trivial]` (*q.v.*).

Change paragraph 4:

- The *object representation* of a complete object type `T` is the sequence of N unsigned char objects taken up by a non-bit-field complete object of type `T`, where N equals `sizeof(T)`. The *value representation* of a type `T` is the set of bits in the object representation of `T` that participate in representing a value of type `T`. The object and value representation of a non-bit-field complete object of type `T` are the bytes and bits, respectively, of the object corresponding to the object and value representation of its type. The object representation of a bit-field object is the sequence of N bits taken up by the object, where N is the width of the bit-field (`[class.bit]`). The value representation of a bit-field object is the set of bits in the object representation that participate in representing its value. Bits in the object representation of a type or object that are not part of the value representation are *padding bits*. For trivially copyable types, the value representation is a set of bits in the object representation that determines a value, which is one discrete element of an implementation-defined set of values. [Footnote: [...] — end footnote]

Trivially copyable types #[basic.types.trivial]

Add this subclause before [basic.fundamental]:

- Each trivially copyable type *T* has an implementation-defined set of discrete *values*. Each possible value representation of an object of type *T* corresponds to a distinct implementation-defined subset of this set. These subsets for a type are disjoint, and their union is the set of values; for scalar types other than object pointer types, each contains no more than one value. Certain operations cause an object to *acquire* a value representation, in which case the object's value is replaced with an unspecified member of the corresponding subset that would result in the program having defined behavior, if any.
- [Note: A single subset for a pointer type can contain pointers to multiple objects in each of several regions of storage whose durations are disjoint. — *end note*]

Move paragraphs 2 and 3 here from [basic.types.general] and change them:

- For any **If an** object ~~(other than~~ **of such a type *T* is not** a potentially-overlapping subobject) ~~of trivially copyable type *T*~~, whether or not the object holds a valid value of type *T*, the underlying bytes ([intro.memory]) making up the object can be copied into an array of `char`, `unsigned char`, or `std::byte` ([cstddef.syn]). [Footnote: [...] — *end footnote*] If the content of that array is copied back into the object, the object ~~shall subsequently hold~~ **acquires** its original value **representation**.

[Example:

[...]

— *end example*]

- For two distinct **such** objects *obj1* and *obj2* ~~of trivially copyable type *T*, where neither *obj1* nor *obj2* is a potentially-overlapping subobject~~, if the underlying bytes ([intro.memory]) making up *obj1* are copied into *obj2*, [Footnote: [...] — *end footnote*] *obj2* ~~shall subsequently hold~~ **acquires** the ~~same~~ value **as representation of** *obj1*.

[Example:

[...]

— *end example*]

#[basic.compound]

Change paragraph 4:

- A pointer value *P* is *valid in the context of* an evaluation *E* if *P* is a null pointer value, or if it is a pointer to or past the end of an object *O* and *E* **happens after the beginning and** happens before the end of the duration of the region of storage for *O*. [...]

#[expr]

#[expr.reinterpret.cast]

Change paragraphs 4 and 5:

- A pointer can be explicitly converted to any integral type large enough to ~~hold~~ **distinguish** all values **representations** of its type. The mapping function is implementation-defined.

[*Note*: It is intended to be unsurprising to those who know the addressing structure of the underlying machine. — *end note*]

[...]

- A value of integral type or enumeration type can be explicitly converted to a pointer. A pointer converted to an integer of sufficient size (if any such exists on the implementation) and back to the same pointer type will have its original value. If the value is equal to that produced by converting one or more pointer values ([basic.compound]); mappings between pointers and integers are to an integral type, the result is an unspecified choice among all such values that would result in the program having defined behavior. If no such value exists, the behavior is undefined.

[*Note*: It is possible for the result to not be valid in the context of the conversion ([basic.compound]), because it points to an object in a region of storage whose duration has ended or has not yet begun. — *end note*]

Otherwise, the result is implementation-defined.

[*Note*: It can be an invalid pointer value. — *end note*]

#[expr.eq]

Insert before paragraph 3:

- Any two pointer values or two pointer-to-member values either compare equal or compare unequal.

[*Note*: Repeated comparisons are consistent so long as neither value is an invalid pointer value. — *end note*]

- If at least one of the operands is a pointer, pointer conversions ([conv.ptr]), function pointer conversions ([conv.fctptr]), and qualification conversions ([conv.qual]) are performed on both operands to bring them to their composite pointer type ([expr.type]). Comparing pointers is defined as follows:
 1. If one pointer represents the address of a complete object, and another pointer represents the address one past the last element of a different complete object, [*Footnote*: As specified in [basic.compound], an object that is not an array element is considered to belong to a single-element array for this purpose. — *end footnote*] the result of the comparison is unspecified.
 2. Otherwise, if the pointers are both null, both point to the same function, or both represent the same address ([basic.compound]), they compare equal.
 3. Otherwise, the pointers compare unequal.

Change paragraph 6:

- If two operands compare equal, the result is true for the == operator and false for the != operator. If two operands compare unequal, the result is false for the == operator and true for the != operator. Otherwise, the result of each of the operators is unspecified.

[*Drafting note*: Which sentence applies might still be unspecified per /3.1, /4.3, or /4.4. — *end drafting note*]

#[bit.cast]

Change paragraph 2:

- *Returns*: An object of type `To`. Implicitly creates objects nested within the result (`[intro.object]`). Each bit of the value representation of the result is equal to the corresponding bit in the object representation of `from`. Padding bits of the result are unspecified. ~~For~~ Every trivially copyable object among the result and each object created within it, ~~if there is no value of the object's type corresponding to~~ acquires the value representation produced; ~~if any such object does not receive a value~~, the behavior is undefined. ~~If there are multiple such values, which value is produced is unspecified.~~ A bit in the value representation of the result is indeterminate if it does not correspond to a bit in the value representation of `from` or corresponds to a bit for which the smallest enclosing object is not within its lifetime or has an indeterminate value (`[basic.indet]`). [...]

Acknowledgments

Thanks to Richard Smith and Peter Sewell for reviewing an early, overcomplicated draft of this paper. Thanks to Hans Boehm and Paul McKenney for an enlightening discussion of [P2414R3](#).